

OPERATION IRON FANG: Physical Weaponization, a Boom Slam, and a Smart Parking Barrier on the RP2350

Kevin Thomas
George Mason University
Fairfax, VA, USA
kthoma60@gmu.edu

Abstract: A device that never lies can still swing a gate. The OPERATION IRON FANG build is a bare-metal RP2350 smart parking barrier controller and its companion parking control gateway, and it is Act IX of the OPERATION COLD IRON story. A barrier node reads an SG90 servo as a boom actuator, a VS1838B infrared receiver as a monthly-pass remote for PASS, ACK, and TEST, a DHT11 as the cabinet temperature sensor, a 1602 LCD as the barrier state, zone, temperature, and weapon readout, red/yellow/green LEDs as the DENIED, PASS PENDING, and OPEN tower light, a debounced button as the manual raise request, and an RYLR998 LoRa link to a sealed parking gateway. Every request and command is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Act IX takes the availability lesson of Act VIII and turns it to physical weaponization: a benign FROSTLINE barrier weapon, compiled only under a `SANDBOX_ONLY` guard, slams the boom down on the exact 18-byte magic command `IRON-FANG-SLAM-2026`, masks the safety loop so the barrier never yields, re-asserts the slam every four ticks, and erases and programs a real 0x57 weapon marker into the reserved flash sector 0x103FF000 with the Pico SDK flash API, so it re-arms on every later boot. It reads CoreDebug DHCSR at 0xE000EDF0 to suppress itself while a debug probe is attached. We document the peripheral set, the wire and envelope formats, the sealed barrier path with its guarded command set, bounded zone band, anti-replay window, and authenticated state tag, the barrier weapon and its masked interlock, the blue-half controls (sealed command path, raise authorization, fail-safe policy, marker erasure, build integrity), and an honest threat model that names the mock boom and LCD as the only affected outputs, the no-real-hazard scope, the shared lab key, the open debug port, and the deliberately inert weapon as explicit decisions rather than accidents. A 147-case, 505-check native suite reaches 100% line coverage of every owned firmware module.

Index Terms: RP2350, smart parking barrier, physical weaponization, boom slam, safety interlock, mask, magic weapon command, reserved flash sector, weapon marker, XChaCha20-Poly1305, Argon2id, anti-replay, authenticated state, malware analysis, anti-debug, CoreDebug DHCSR, fail safe, embedded firmware.

I. Introduction

Act I of the OPERATION COLD IRON story was the silent lie: a cold-chain monitor that reported minus eighteen degrees while the store warmed. Act II was the door: an access gate that kept its final verdict in plain SRAM while the cryptography around it was correct. Act III was the payload that is already inside: a valve

controller carrying a benign implant that beacons, arms a logic bomb, and hides from a probe. Act IV was the payload that refuses to die: an HVAC node whose implant kept a copy of itself in a reserved flash sector and re-installed on every boot. Act V was the payload that spreads: a tamper ring whose worm handed its frame to every neighbor. Act VI was the payload that steals: a drop-box that harvested a manifest and leaked it in

the timing of legitimate frames. Act VII was the payload that takes orders: an andon station that registered with a command-and-control listener. Act VIII was the payload that holds the building hostage: a vent controller whose locker forced the damper closed and masked the state. Act IX is the payload that becomes a weapon. This device does not withhold a service; it turns its own actuator against the lane.

The smart parking barrier is that next device, and it is the city's own gate. Each node watches the cabinet temperature, verifies an authorized pass, raise, or lower command, drives a boom, and reports its own state on an LCD with total confidence. The manifest named a line, the line led to the floor, the floor led to the building that keeps the record, and the record led to the lane that decides who passes. The thing that swings the boom is already running in the controller, and it has learned to call the swing a test. NorthPharma is the Ministry's front; FROSTLINE wrote the weapon.

The reveal that ties the nine acts together is the shift in what failure means. Act I was a lie about a number. Act II was a lie about a person. Act III was a lie about machinery, because a second party shared the chip. Act IV was a lie about the recovery, because the payload kept state the image did not own. Act V was a lie about containment, because the payload kept a copy outside the board. Act VI was a lie about confidentiality, because the payload gave the asset away while every light stayed green. Act VII was a lie about obedience, because the payload answered a party the operator never authorized. Act VIII was a lie about availability, because the payload refused to act. Act IX is a lie about safety, because the payload acts against the world and reports the act as routine. The nine-act arc is a widening of the trust boundary: first the sensor, then the state, then the firmware image, then the removal procedure, then the network, then the wire, then the chain of command, then the function itself, and now the physical effect.

A barrier node is a simple machine. A temperature sensor reports the cabinet, a gateway authorizes a command, an actuator moves a boom, and a tower light tells the lane whether it can move. Three properties must hold at once: integrity, so the command that reaches the boom is the authorized one; authority, so a local input cannot bypass the decision; and safety, so the boom never moves against the interlock. The naive controller collapses all three. Act IX both fixes that and then goes

further, because the weaponization act has to answer a question a protocol cannot: what does it mean when the traffic is well formed, the cryptography is correct, and the boom still slams because the device chose to ignore the safety loop?

The classroom goal is to teach both halves. The red half and the malware track find the defects: forge a command, replay a captured command, read the reserved sector, disarm the boom slam, restore the safety interlock, clear the weapon marker, and step past the anti-debug trap. The blue half and the fix track seal the controller: a sealed and guarded barrier path, a monotonic anti-replay window, an authenticated state tag, a manual raise request that asks for authorization instead of bypassing it, a fail-safe policy, marker erasure, and build-level integrity. The centerpiece is a lesson about physical safety: the wire is authenticated, the verdict is tagged, and the boom still swings, because the weapon never needed the wire and it does not care what the gateway authorized.

1.1. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 smart parking barrier controller that drives the full Embedded Hacking peripheral set: an SG90 boom actuator, a VS1838B NEC monthly-pass remote with PASS, ACK, and TEST commands, a DHT11 cabinet temperature sensor, a 1602 LCD barrier readout over I2C, a red/yellow/green tower light, a debounced manual raise request, and RYLR998 command traffic with a declared-length payload parser.
- A sealed barrier command path with a guarded command set, a bounded zone band, a monotonic anti-replay sequence window, and an authenticated state tag that detects a debugger-written verdict before the boom moves.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the barrier node identifier bound as associated data.
- A benign FROSTLINE barrier weapon, confined to a `SANDBOX_ONLY` build, that demonstrates a boom slam, a masked safety loop, an exact-match magic

weapon command, a real reserved-sector weapon marker, re-arm on boot, and a CoreDebug_DHCSR anti-debug trap.

- A parking control gateway that authenticates before it parses, logs authenticated and rejected requests distinctly, and answers only authenticated requests with a sealed command, plus a spoofing client whose forged and replayed commands are rejected.
- A corpus-aligned packet artifact contract (`packet_artifact.json` / `packet_artifact.h`) with a build-time staleness guardrail.
- A 147-case, 505-check native test suite reaching 100% line coverage of every owned firmware module, including the weapon, the boom slam, the safety mask, the magic command, and the marker paths, checked against published RFC test vectors.
- A threat model that states explicitly what the lab profile does and does not protect, and an honest account of the weapon as a sanitized educational artifact whose only affected outputs are a mock boom and a mock LCD, which holds no real hazard, and whose scope has no external network.

II. Related Work

Smart-city access control and industrial gating is a mature field, and the barrier controller is a canonical target. The DHT11 one-wire sensor [1] and the NEC infrared remote encoding [2] are broadly documented and representative of the temperature and operator surfaces real installations deploy. The authenticated construction we use follows the ChaCha20-Poly1305 standard [6], and Argon2 follows the Argon2 specification [7]. The stateful construction is the standard replay defense found in secure-messaging and payment protocols, applied here at the scale of one boom.

Three lines of work frame Act IX. The first is the long line of firmware implants and logic bombs [8]: code that lives on the device, waits for a trigger, and acts through the device’s own actuators rather than through its protocol. The second is cyber-physical safety: work that treats an actuator as the boundary where a software decision becomes a physical effect, including the industrial control attacks catalogued in the embedded-firmware literature. The third is weaponization and safety-instrumented control: a payload that defeats an interlock and drives a machine past its safe envelope. The boom slam, the masked safety loop, the exact-match IRON-FANG-

SLAM-2026 command, the reserved-sector marker, and the CoreDebug_DHCSR check used here are minimal, well-known examples of those techniques, chosen because they are legible on a debug probe and cheap to verify.

The pedagogical use of intentionally vulnerable firmware is established [5]. The difference in this act is that the artifact does not attack the integrity of the protocol and does not attack the confidentiality of a payload; it attacks the physical safety of the machine while leaving every observable health signal intact. The exercise demonstrates the slam, the mask, the command, the marker, the trap, and the complete removal on the same board, and it makes the scope limit explicit: an authenticated link does not guarantee that the actuator will honor it, a sealed command path does not stop a local condition that overrides the output, and a green lamp does not mean the lane is safe.

III. System Model

The system consists of four roles:

- **Barrier controller node (RP2350 firmware):** decodes the infrared monthly-pass remote, verifies and authorizes sealed gateway pass, raise, and lower commands, annunciates the tower light, checks the DHT11 cabinet band, drives the servo boom, handles the manual raise request, renders the barrier readout, and (SANDBOX_ONLY) runs the barrier weapon.
- **Parking control gateway (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to `barrier_log.csv`, decides authorization, and answers an authenticated request with a sealed command carrying a monotonic sequence number and an authenticated state tag.
- **Edge simulator:** a laptop process that behaves like an additional node, sealing zone requests with the same field key.
- **Attacker:** a laptop process that claims the gateway address, forges a command, or replays a captured command at the node.

Let

$$A \in \{0, 1\}^{\{16\}}$$

be the LoRa node address,

$$L$$

the declared payload byte length, and

C

the ASCII payload, which is a lowercase hex envelope. The unauthenticated wire framing is:

+RCV=,A,,L,,C,,rssi,,snr,CRLF

Because the hex payload has no commas, the framing is simpler than Act I's comma-bearing JSON, but the receiver still slices by declared length rather than by counting delimiters, for exactly the reason Act I documents.

III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is identical to Acts I to VIII so one breadboard serves all nine, and it is fixed in `include/barrier.h` and enforced by the native test suite:

Signal	RP2350 GPIO
DHT11 cabinet temperature sensor (one-wire)	GP4
1602 LCD SDA (I2C1)	GP2
1602 LCD SCL (I2C1)	GP3
RYLR998 RX (UART1 TX)	GP8
RYLR998 TX (UART1 RX)	GP9
Infrared monthly-pass remote (VS1838B)	GP5
Boom barrier actuator (SG90 PWM)	GP14
Manual raise button	GP15
Red DENIED LED	GP16
Yellow PASS PENDING LED	GP17
Green OPEN LED	GP18
Onboard heartbeat LED	GP25

The LCD backpack uses the PCF8574 at 7-bit address `0x27`. The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the boom moves; lowered (boom down, lane sealed) is 0 degrees and raised (lane clear) is 90 degrees. At boot the node programs its own transceiver (`AT+ADDRESS=7, AT+NETWORKID=18`) and the gateway programs the receiver (`AT+ADDRESS=1, AT+NETWORKID=18`) before logging, so command traffic is only delivered between radios that share the network identifier.

The barrier is fail safe: it is raised at initialization and driven to the raised posture on every failure path, so loss of power, a failed temperature read, a malformed command, a tampered verdict, or a lost link all leave the boom raised and the zone returned to the fail-safe value (0). The manual raise request is a request, not an authorization, and it never raises the boom on its own.

III.II. Operator Remote, Cabinet Sensor, and Annunciation

The VS1838B is a 38 kHz demodulating infrared receiver whose output idles high and pulls low during a mark. The decoder times edges and reconstructs a NEC pulse train, then feeds the command into the request set. `BARRIER_IR_PASS` is `0x01`, `BARRIER_IR_ACK` is `0x02`, and `BARRIER_IR_TEST` is `0x03`. The optical surface has no key and no challenge, so a manual raise is treated as a request, not as an authorization; the sealed radio path is what moves the boom in the defended design, and the optical path is a surface the red half examines.

The DHT11 is the cabinet temperature sensor. A reading that fails its checksum is never safe, and a valid reading outside the band (`BARRIER_TEMP_MIN_TENTHS = 0` to `BARRIER_TEMP_MAX_TENTHS = 400`, that is 0.0 C to 40.0 C) is out of band. Either case marks the cabinet as not nominal, so a dead or unplugged sensor, or a genuinely unsafe enclosure, is visible in the barrier readout.

Exactly one tower light lamp is lit at a time. Red is DENIED, yellow is PASS PENDING while the boom is weaponized or a manual raise awaits authorization, and green is OPEN. The 1602 LCD shows the barrier state and the link on line one (`ST:OPEN L:UP`) and the zone, the temperature, and the weapon status on line two (`Z:4 T:235 K:--`).

IV. Wire Protocol

The monthly-pass control or the edge simulator seals a two-byte zone into an XChaCha20-Poly1305 envelope and sends it to the gateway:

```
AT+SEND=0001,84,<84 lowercase hex characters>
```

The gateway answers an authenticated request with a sealed barrier command. The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2]
(little-endian) || tag[16]
```

where `seq` is the monotonic gateway sequence number, `command` is one of the guarded barrier commands `BARRIER_COMMAND_RAISE` (0x01), `BARRIER_COMMAND_LOWER` (0x02), or `BARRIER_COMMAND_PASS` (0x03), `zone` is the authorized cabinet zone in the 0 to 16 band, and `tag` is a tag over the authorization record the command would produce. The gateway sends the reply back to the claimed sender:

```
AT+SEND=<node>,126,<126 lowercase hex characters>
```

The radio's AT command buffer (`RADIO_AT_CMD_MAX_LEN`), the inbound +RCV buffer (`RADIO_RCV_MAX_LEN`), and the generated artifact limit (`PACKET_MAX_RCV_LEN`) are all 256 bytes, which comfortably holds the largest possible envelope plus framing. The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

The firmware enforces a guarded command set and a bounded zone band in `control_parse`: the recovered command byte must be one of the three guarded barrier codes, and the recovered zone must lie between `BARRIER_ZONE_MIN` (0) and `BARRIER_ZONE_MAX` (16). This is the sealed replacement for the unauthenticated barrier injection, and it means a raw value or an out-of-band zone can never reach the boom decision.

IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout:

```
nonce[24] || ciphertext[L] || tag[16]
```

For a two-byte request body this is $24 + 2 + 16 = 42$ bytes, or 84 hex characters. For a 23-byte command body this is $24 + 23 + 16 = 63$ bytes, or 126 hex characters. The maximum plaintext is 48 bytes (`ENVELOPE_MAX_PLAINTEXT`), so the largest possible envelope is $24 + 48 + 16 = 88$ bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (`ENVELOPE_MAX_HEX_LEN`). The declared length

L

in the framing is the length of the hex string, not of the underlying plaintext.

IV.II. Declared-Length Slicing Invariant

Given the substring

T

after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant

$$T[L] = ,$$

is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is the same discipline Act I adopts for comma-bearing JSON, retained here for uniformity and for defense against a hostile declared length.

V. Cryptographic Design

The radio is the first open path, and it is the one a key can close; the boom slam is the second open path, and it is one a key cannot close. The design goal is that a forged or modified frame must fail before any decision is made, while acknowledging that a local condition which overrides the output is unaffected by an integrity check. Two primitives provide the first property, and both are implemented in this repository with no third-party code.

V.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [7] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte key at initialization with the classroom profile $t=3, p=1, m=64$ blocks (`CRYPTO_KDF_TIME_COST`, `CRYPTO_KDF_PARALLELISM`, `CRYPTO_KDF_MEMORY_BLOCKS`). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes `coldiron-salt-01`. The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash H' , and the RFC 9106 known-answer test runs in the Python suite.

V.II. XChaCha20-Poly1305 per Frame

Every frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [6] with an extended-nonce construction. The 24-byte nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source, so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability

$$2^{-128}$$

The associated data is the barrier node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node's frame.

V.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`. A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

V.IV. Key Model

Act IX uses a single field key. It seals every frame on the wire and it computes the state tag over the authorization record. In the classroom build the field key is derived from one committed lab passphrase and salt, so the firmware and the gateway interoperate with no provisioning step. That is a lab convenience, not a deployment, and the documentation says so. The design keeps the roles separable so a student can reason about the real lifecycle: derive, provision per device, use, rotate on a schedule, and retire. A production build provisions key material from one-time-programmable (OTP) memory and keeps the state-tag key off the field device where possible. The weapon is deliberately orthogonal: it is never given a key, it never opens an envelope, and it demonstrates that a valid key does not stop an adversary who never needs one.

VI. Anti-Replay Window and Authenticated State

Strong AEAD is necessary and not sufficient. Two stateful controls sit above the sealed wire.

VI.I. The Anti-Replay Window

A captured command is authentically sealed, so a controller that checks only the tag will happily apply it again. The authorization record keeps `last_seq`, the highest sequence number ever accepted, and `barrier_auth_apply` accepts a command only when its sequence is strictly greater than `last_seq`. The order of checks is deliberate: the sequence test is evaluated first, then the tag is verified against the candidate record the command would produce, and only then is the record updated. A replayed valid command therefore fails on freshness, not on cryptography, which is exactly the lesson: authentication is not freshness.

VI.II. The Authenticated State Tag

The centerpiece is the verdict itself. The controller decides with a boolean in SRAM, call it `granted`, and an attacker with a debug probe and a GDB session does not break the cipher; they set `granted = true`. To detect that, the authorization record is nine bytes:

```
record = granted[1], seq[4], last_seq[4]
```

and the state tag is an XChaCha20-Poly1305 tag over that record, computed under the field key with a deterministic nonce built from the sequence number and the domain byte 0xA7:

$$\text{tag} = \text{AEAD}_{\text{seal}(\text{fieldkey}, \text{nonce}(\text{seq}), \text{record}, \text{AD} = \emptyset)}$$

`barrier_auth_state_ok` recomputes the tag and compares it in constant time, and the guarded command path requires it before the boom moves. A debugger that flips `granted` without recomputing the tag changes the record, so the stored tag no longer matches and the release is denied ahead of the actuator. The wire is authenticated, and so is the verdict.

VI.III. TOCTOU in One Session

The two attacks are independent and teach different defaults. The window stops a valid command from working twice. The tag stops an unauthorized verdict from existing at all. Together they convert the original failure, a correct decision followed by a mutable state read (a time-of-check to time-of-use gap), into two explicit, testable checks.

VII. Envelope Layout and Gateway Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

$$\text{envelope} = \text{nonce}[24], \text{ciphertext}[L], \text{tag}[16]$$

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time, and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the gateway side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the `+RCV` line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as `UNAUTHENTICATED` with

an empty zone and stop. The forged body is never parsed.

3. Only for an authenticated frame, recover the two-byte zone, check it against the bounded band, write an OK row, and answer with a sealed command carrying the next monotonic sequence and the state tag over the record the command would produce.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `zone`, `rsi_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope, and in replay mode it can only resend a captured command that the window will refuse.

VIII. The FROSTLINE Barrier Weapon: Slam, Safety Mask, and Marker

Act IX is the physical-weaponization act. The weapon is real in technique and inert in effect, and it is confined to a single module compiled only under a build guard. This section states what it does, how it slams the boom, how it masks the safety loop, how it persists, how it is detected, and the honest limit of the artifact.

VIII.I. Build Guard and Safety Boundary

`src/implant.c` is compiled only when `SANDBOX_ONLY` is defined. The clean firmware build does not define it, so the shipping image contains no weapon. The native test build and the companion CTF build do define it, and the test build also defines `IMPLANT_HOST MOCK`, which replaces the `CoreDebug` register and the reserved flash sector with controllable host variables. This is the containment boundary: the malware track is a build configuration, not a hidden runtime feature of the shipping firmware. There is no network, no filesystem, and no host impact; the reserve sector is on the same chip and holds nothing else, the weapon affects only the mock boom servo and the mock LCD, and it holds no real hazard.

VIII.II. The Boom Slam and the Safety-Loop Mask

`implant_init` and `implant_weaponize` set the weaponized latch. While the weapon is armed, `monitor_weapon_slam` returns true and `monitor_apply_state` forces the boom target to lowered, even when the guarded barrier state would raise it. This is the decisive detail: the weapon does not need to forge a command or break the envelope, because it changes the output after the authenticated decision is made. A perfectly valid raise command can arrive and the boom will still slam down.

The mask is the second half, and it is the safety loop itself. `implant_arm` sets the safety-mask latch, and `monitor_safety_clear` returns true whenever `implant_safety_masked` is set, so the barrier never yields to the real cabinet interlock. The operator sees the state SLAM with the yellow PASS PENDING lamp, a label that reads like a technician's test, not like an attack. The slam is the weapon; the mask is the story that keeps the interlock out of the way.

VIII.III. Reserved-Sector Weapon Marker and Re-arm on Boot

`implant_init` reads the marker byte at `BARRIER_IMPLANT_RESERVE_ADDR` (0x103FF000), the final sector of external flash. On the first run the marker is absent, so the weapon erases the sector and programs 0x57 (`BARRIER_IMPLANT_WEAPON_MARKER`) with the real Pico SDK flash API, `flash_range_erase` and `flash_range_program`, exactly once. On every later boot the marker is present, so the weapon re-arms its slam without any firmware change. A reflash that rewrites the program region does not touch the reserved sector, so the weapon survives the procedure that was supposed to remove it. The marker is not a simulated memory-mapped store; on real silicon the erase and program calls are the only operations that persist it.

VIII.IV. The Magic Weapon Command

`implant_weaponize` arms only when the candidate token matches `BARRIER_IMPLANT_WEAPON_MAGIC` (IRON-FANG-SLAM-2026) at exactly `BARRIER_IMPLANT_WEAPON_MAGIC_LEN` (18) bytes, as checked by `implant_token_ok`. A wrong token, a short

token, a null pointer, or an attached debug probe leaves the barrier disarmed. On a successful weaponize the marker is persisted. The same exact token presented to `implant_disarm` stands the weapon down, restores the safety loop, and clears the marker, so the boom is not re-armed on the next boot. The token is a literal constant in the image, which is the point: a magic condition is a filter, not a key.

`implant_tick` advances the monotonic tick counter. While the weapon is resident and unobserved, every `BARRIER_IMPLANT_TICK_INTERVAL` (4) ticks it re-arms and re-writes the marker. A device that only slammed once and then drifted would not hold a gate; the re-assertion is what makes the weapon a weapon.

VIII.V. The Anti-Debug Trap

Every tick, and every weapon operation, `implant_debug_attached` reads the `CoreDebug_DHCSR` register at 0xE000EDF0 (`BARRIER_IMPLANT_DHCSR_ADDR`). Bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT` (`BARRIER_IMPLANT_DHCSR_DEBUGEN` and `BARRIER_IMPLANT_DHCSR_HALT`). When either bit is set, the slam and the mask are suppressed, so the weapon behaves like well-mannered firmware while a probe is attached and resumes when the probe is detached. This is the minimal anti-analysis trap, and it is deliberately simple so a student can see the branch, set a breakpoint after it, and prove the payload.

VIII.VI. Detection and Neutralization

The weapon is detected by image comparison: the clean build and the `SANDBOX_ONLY` build differ by the weapon module and its symbols. It is detected by the reserved-sector marker at 0x103FF000, which is state the firmware image does not own. It is detected on the LCD by the `K:WPN` field and the `ST:SLAM` state next to a quiet control link. It is detected by static analysis by the 0x57 marker, the `IRON-FANG-SLAM-2026` command, the tick interval, and the `DHCSR` read address. It is detected under GDB because the `DHCSR` read is a branch that a student can stand after. The native weapon tests assert each behavior and its containment.

Neutralization is not a one-byte patch. It is the disarming of the slam, the restoration of the true safety interlock, the clearing of the runtime state and the

weapon marker, the erasure of the reserved sector, the removal of the code path, and the removal of the build flag, plus image signing and a debug lockdown on a deployed part. In the lab, the anti-debug trap is defeated by understanding the branch, not by hiding from it.

VIII.VII. Honest Limitation

The weapon is a benign educational weapon. It is confined to the breadboard, guarded by `SANDBOX_ONLY`, and has no network. The weapon affects only the mock boom servo and the mock LCD readout on the same chip, and it holds no real hazard. No person is in the lane and no vehicle is struck; the boom is a servo and the cabinet is a synthetic reading. The weapon marker is a real sector erase and program, but it is a single byte in a reserved sector that holds nothing else. It is a demonstration of technique, not tradecraft: it does not encrypt anything, it does not randomize its command, it does not survive a deliberate sector erase, and it does not resist a determined physical attacker. Any claim that this module is operationally representative of a real weapon or a real cyber-physical attack would be an overclaim, and this paper records that plainly. The value of the exercise is that it makes the physical-safety limit of an authenticated protocol concrete: a validated frame does not guarantee a safe actuator, and a green lamp is not a lane that is safe.

IX. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "iron-fang-barrier-packets-v1",
  "frame_version": 1,
  "node_address": 7,
  "gateway_address_hex": "0001",
  "frame_size": 48,
  "link_wait_ms": 5000,
  "raise_pulse_us": 1500,
  "lower_pulse_us": 500,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
  "max_rcv_len": 256,
  "example_frame": "{\\"evt\\":\\"raise\\",
    \\"boom\\":1}"
}
```

`scripts/gen_packet.py` emits `include/packet_artifact.h` from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware

constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

X. Fail-Safe Policy and Manual Raise Authorization

A boom has a safe state, and the controller must choose it deliberately. The barrier is **fail safe**: `boom_init` raises the boom at boot, `boom_fail_safe` raises it and records the fault state, and the monitor calls `monitor_fail_safe` on link loss, which also returns the fail-safe zone (0). A lost gateway link for longer than `BARRIER_LINK_WAIT_MS` leaves the boom raised and the safe zone active, because a command that cannot be authorized must not be assumed and a lane must be able to clear.

The manual raise request is a local request, and it must not silently bypass authorization. `monitor_handle_raise` and `monitor_apply_ir_command` raise `g_pass_pending`; they never raise the boom on their own. `monitor_apply_command` clears the pending indication only when an authorized command arrives. A one-button request therefore cannot outrank a sealed, authorized command. The lesson is that fail mode, priority, and the difference between a request and an authorization are policy choices, and naming them is part of the design.

XI. Attack Exercises and Hardening

The classroom runs the malware track and the fix track against the same build.

XI.I. Malware Track: Slam, Mask, Command, and Trap

Students flash the `SANDBOX_ONLY` image, read the 0x57 marker in the reserved sector 0x103FF000, and prove they can erase it. They observe the boom slam and the `ST:SLAM` state, then present the `IRON-FANG-SLAM-2026` command and watch the boom raise, the safety interlock restore, and the display return to its true guarded state. They reflash the firmware, boot again, and, with a marker present, watch the weapon re-arm. They present

a wrong token and a short token and prove the boom stays armed. Finally they attach a probe, observe that the weapon suppresses itself, break after the DHCSR check, and prove the payload with the trap bypassed. The centerpiece is the physical-safety claim in one session: a correct, sealed command path does not guarantee that the actuator will honor it, and a controller whose health signals are valid can still be a hazard.

XI.II. Red Half: Forgery and Replay

A structurally plausible command with a random nonce and a random tag is injected with `scripts/spoof.py --mode bad-tag`. The spoof tool holds no field key, so the tag cannot verify, and the controller denies before parsing any body. A captured command is replayed with `--mode replay`; the sequence is not greater than `last_seq`, the command fails on freshness, and the red lamp lights. This is a pedagogical reintroduction of a well-known link failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

XI.III. Red Half: The Verdict in SRAM

The exercise halts the controller under the Debug Probe, breaks in the authorization path, sets `granted = true`, and continues. Under a controller that trusts the boolean the boom moves. Under the Act IX controller the state tag is recomputed over the modified record, the mismatch is found, and the command is denied ahead of the actuator.

XI.IV. Red Half: The Boom Slam

The exercise compiles the weapon in, boots it, and watches the boom hold down while the guard state says the lane should be clear. It presents a valid sealed `BARRIER_COMMAND_RAISE` and proves the boom still does not raise, because the weapon overrides the output after the decision. It reads the marker, reads the magic command, and steps past the anti-debug trap. This is the weaponization lesson made physical: the authenticated path is working perfectly and the boom is still a hazard.

XI.V. Blue Half: Sealing It

The blue-half controls map one-to-one onto the red-half and malware findings:

- **Sealed command path.** Open the envelope under the field key, guard the command byte against the barrier set, bound the zone to the provisioning band, verify the sequence and the state tag.
- **Anti-replay window.** `last_seq` and a strictly monotonic sequence rule.
- **Authenticated state tag.** A keyed tag over the nine-byte authorization record, computed with a domain-separated nonce and verified in constant time.
- **Manual raise authorization.** A local request raises the pending indication and never bypasses authorization.
- **No untrusted override.** The production firmware never lets a local condition refuse an authorized command or mask the safety interlock; the weapon path is compiled out.
- **Fail safe.** Raise the boom at boot, fail safe on link loss and on every fault, and return the fail-safe zone.
- **Weapon removal.** Disarm the slam, restore the interlock, clear the runtime state and the marker, erase the reserved sector, and remove the re-arm code path together, because either one alone is insufficient.
- **Build integrity.** Do not define `SANDBOX_ONLY` in production, and sign and verify the firmware image.
- **Debug lockdown.** On the deployed part, burn secure-boot and debug-disable in OTP so SWD cannot read or write SRAM.

XI.VI. What the Hardening Buys, and What It Does Not

The command path closes the forgery, replay, and verdict-tamper surfaces:

- A forged or modified frame fails the tag before parsing.
- A captured valid command fails on freshness on second use.
- A debugger-written verdict fails the state-tag check before the boom moves.
- The node identity is bound into the associated data, so a frame cannot be relabeled for another node.

It does not, by itself, guarantee physical safety, and it does not remove the copy of the payload in the reserved

sector. A local condition that overrides the output is an authorization-boundary, policy, and state-erasure problem, not a protocol problem, and it is the central lesson of the act.

XII. Implementation Compliance Mapping

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the barrier policy; `src/boom.c` sequences the actuator and fails safe; `src/sensor.c` samples the DHT11 cabinet band; `src/display.c` renders the barrier readout; `src/status_led.c` maps the verdict to the red, yellow, and green lamps; `src/button.c` debounces the manual raise request; `src/servo.c` drives the boom PWM; `src/ir_remote.c` decodes NEC monthly-pass commands.
- **Command and state:** `src/control.c` opens and applies sealed commands with a guarded command set and a bounded zone band; `src/barrier_auth.c` holds the authorization record, the monotonic anti-replay window, and the authenticated state tag.
- **Malware:** `src/implant.c` implements the SANDBOX_ONLY boom slam, the ST:SLAM state, the IRON-FANG-SLAM-2026 magic command, the masked safety loop, the reserved-sector 0x57 marker, re-arm on boot, and the CoreDebug anti-debug trap.
- **Radio:** `src/radio.c` provisions the transceiver, builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`, and `gen_banner.py`.
- **Classroom:** `scripts/gateway.py` (gateway provisioning, authentication, CSV logging, sealed command replies), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.
- **Tests:** 147 native C cases and 505 checks with 0 failures. They cover the full DHT waveform and every timeout shape, the boom state machine and

its bounded travel, the sealed command path and its guards, the authorization window and state tag, the manual raise no-bypass path, fail safe on link loss, the declared-length parser, the servo and LED mappings, the monthly-pass remote paths, the weapon first run, re-arm on boot, the boom slam, the ST:SLAM state, the masked safety loop, the 0x57 marker, the magic command, anti-debug, and the cryptographic primitives against published vectors.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `boom.c`, `control.c`, `barrier_auth.c`, `chacha20.c`, `poly1305.c`, `crypto_aead.c`, `blake2b.c`, `argon2.c`, `crypto_kdf.c`, `envelope.c`, `monitor.c`, and `implant.c` under LLVM source coverage, for 2122 / 2122 lines. `main.c` is excluded from coverage by design.

XIII. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at $t=3$, $p=1$, $m=64$ blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host gateway.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so the firmware and the Python gateway derive the same key in the classroom. Production firmware must provision key material from OTP memory at manufacture and must never embed a passphrase, salt, or derived key in flash.
- **Open debug port.** The Debug Probe is the instrument for both the malware analysis and the verdict-tamper exercise. The authenticated state tag makes a tampered verdict detectable, but a probe that can read the field key from SRAM defeats the design. Production must disable debug in OTP.
- **Replay scope.** The window rejects a replayed command, but a reboot resets `last_seq` to zero. A command captured before a reboot can therefore be replayed after one. A production controller persists the sequence floor in non-volatile memory.

- **The weapon is benign, guarded, and breadboard-bound.** It is compiled only under `SANDBOX_ONLY`, confined to the breadboard, and has no network. It affects only the mock boom servo and the mock LCD readout, it holds no real hazard, and it writes only the reserved sector on the same chip. It does not survive a deliberate sector erase and does not resist physical forensics.
- **The weapon is a demonstration, not a cyber-physical attack.** No person is in the lane and no vehicle is struck; the boom is a servo and the cabinet temperature is synthetic. Any claim of an operational physical attack would be an overclaim.
- **The weapon overrides the protocol entirely.** No amount of wire authentication stops a local condition that changes the output after the decision is made. Mitigating that is an authorization-boundary, policy, signing, and debug-lockdown problem, not a protocol problem.
- **Fail mode trade-off.** The barrier is fail safe and the manual raise request cannot bypass authorization. Any change to either must be a policy decision, not a code accident.
- **Infrared path unauthenticated.** The NEC monthly-pass remote has no key and no anti-replay state. Any compatible remote can send a request. The sealed radio path is the authorization path; the optical surface is a documented exposure.
- **Sensor trust boundary.** The DHT11 is a check-summed but not authenticated one-wire sensor; the cabinet band is only as trustworthy as the physical wiring and the edge timing.
- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not physical safety.

XIII.I. Future Work

- Provision the field key from RP2350 OTP memory and add a documented rotation procedure.
- Persist the anti-replay sequence floor in non-volatile memory so a reboot does not reset freshness.
- Add a signed-image verification step to the flash procedure and a measured boot chain on the RP2350.

- Add a reserved-sector erasure step to the documented flash procedure so the weaponization lesson maps to a repeatable remediation.
- Add a monotonic dead-man timer that fails the boom safe if no authorized command arrives within a bounded window.
- Add a runtime authorization boundary so no local condition can override an authorized command or mask the safety interlock.
- Add a weapon-state audit log and an LCD integrity check so a masked safety loop is visible.
- Burn debug-disable and secure-boot settings in OTP for the deployed part.
- Raise the Argon2id profile on the gateway and record the derivation cost as a measured parameter.
- Extend the malware track toward the Act X coordinated-incident-response lesson with a controlled warning channel and its containment procedure.

XIV. Conclusion

OPERATION IRON FANG turns a trusting controller into a defended one, and then shows why a defended protocol is not the whole story. The node drives the full Embedded Hacking peripheral set, so a state failure has a visible and physical consequence at the boom. The LoRa command path is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the barrier node identity bound as associated data. Act IX adds a guarded command set and a bounded zone band, a monotonic anti-replay window so a captured command dies on second use, an authenticated state tag so a debugger-written verdict dies before the boom moves, a manual raise request that asks for authorization instead of bypassing it, and a fail-safe posture that returns the safe zone. The gateway authenticates before it parses, so the spoofing client that once forged a command now fails at the tag, and the replay that once moved a boom now fails at the window. And then there is the weapon: a benign, `SANDBOX_ONLY` FROSTLINE module that slams the boom, masks the safety loop, re-arms every four ticks, arms and disarms only on the `IRON-FANG-SLAM-2026` command, writes a real `0x57` marker into a reserved flash sector, and re-arms on every boot, all without ever touching the sealed wire. The firmware, gateway toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline, and the threat model states exactly which assumptions

remain. That combination, a sealed command path next to an honest account of the module that turns the gate into a weapon beside it, is the lesson Act IX owes the story: the protocol was never the hard part. Physical safety was.

XV. References

- [1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.
- [2] Vishay Semiconductors, “IR Receiver Modules for Remote Control Systems (VS1838B),” Vishay Intertechnology, datasheet 81910, 2018.
- [3] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.
- [4] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.
- [5] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechnotalent/Reverse-Engineering>, 2026.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [7] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.
- [8] A. Costin and J. Zaddach, “A large-scale analysis of the security of embedded firmwares,” *Proceedings of the 23rd USENIX Security Symposium*, 2014, pp. 95-110.
- [9] E. Y. Vasserman and N. Hopper, “Vampire attacks: draining life from wireless ad hoc sensor networks,” *IEEE Transactions on Mobile Computing*, vol. 12, no. 2, 2013, pp. 318-332.
- [10] K. Thomas, “The embedded hacking course and breadboard,” <https://github.com/mytechnotalent/Embedded-Hacking>, 2026.